

events.c

```
#include <stdio.h>
#include <stdlib.h>
#include "types.h"
#include "functions.h"

/* Keeps the loan interest rate inside its allowed range. */
void clampLoanInterest(GameState game[])
{
    if(game[0].economy.loanInterestRate < LOAN_INTEREST_MIN)
        game[0].economy.loanInterestRate = LOAN_INTEREST_MIN;

    if(game[0].economy.loanInterestRate > LOAN_INTEREST_MAX)
        game[0].economy.loanInterestRate = LOAN_INTEREST_MAX;
}

/* Moves the loan interest rate up or down by deltaPercent. */
void adjustLoanInterest(GameState game[], int deltaPercent)
{
    game[0].economy.loanInterestRate += deltaPercent;
    clampLoanInterest(game);
}

/* Appendix A: 20 National Event Cards, drawn when a player lands on
an EVENT square. currentCardIndex cycles 0-19 and wraps around,
which behaves the same as "draw the top card, put it at the
bottom" without needing a real deck data structure. */
void executeEvent(GameState game[], int playerIndex)
{
    (void)playerIndex; /* not needed - these effects are global, not per-player */

    printf("\n*** NATIONAL EVENT CARD ***\n");

    switch(game[0].economy.currentCardIndex)
    {
        case 0:
            printf("Tourism Hype : Hotels earn double rent for 5 rounds.\n");
            addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 5);
            break;

        case 1:
            printf("Fuel Shortage : Railway rent doubles for 5 rounds.\n");
            addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 5);
            break;

        case 2:
            printf("Heavy Floods : a random coastal property is damaged.\n");
            triggerDisaster(game);
            break;

        case 3:
        {
            int candidates[BOARD_SIZE];
            int count;
            int i;

            count = 0;

            for(i = 0; i < BOARD_SIZE; i++)
            {
                if(game[0].board[i].type == PROPERTY && game[0].board[i].property.owner != -1)
```

```

        {
            candidates[count] = i;
            count++;
        }
    }

    if(count > 0)
    {
        game[0].economy.closedPropertyIndex = candidates[rand() % count];
        game[0].economy.closedPropertyRoundsLeft = 2;

        printf("Political Rally : %s is closed for 2 rounds.\n",
            game[0].board[game[0].economy.closedPropertyIndex].name);
    }
    else
    {
        printf("Political Rally : no owned property to close.\n");
    }
    break;
}

case 4:
    printf("Stock Market Rise : All property values increase by 10%% for 15 rounds.\n");
    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);
    break;

case 5:
    printf("Economic Downturn : Property values decrease by 15%% for 15 rounds.\n");
    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
    break;

case 6:
    printf("Housing Subsidy : House construction cost reduced by 30%% for 15 rounds.\n");
    addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 15);
    break;

case 7:
    adjustLoanInterest(game, -LOAN_INTEREST_STEP);

    printf("Interest Rate Cut : Loan interest reduced by %d%%. Now %d%%.\n",
        LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
    break;

case 8:
    adjustLoanInterest(game, LOAN_INTEREST_STEP);

    printf("Interest Rate Increase : Loan interest increased by %d%%. Now %d%%.\n",
        LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
    break;

case 9:
{
    int i;

    printf("Tax Amnesty : Each player receives LKR 2,000.\n");

    for(i = 0; i < MAX_PLAYERS; i++)
        receiveMoney(game, i, 2000);

    break;
}

case 10:

```

```

        printf("Power Failure : Utility income halved for 3 rounds.\n");
        addModifier(game, MOD_UTIL_RENT, -1, -1, 50, 3);
        break;

case 11:
    printf("Foreign Funding : Commercial property values increase by 15%% for 15 rounds.\n");
    addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 115, 15);
    break;

case 12:
    printf("Port Expansion : Railway station values increase by 20%% for 15 rounds.\n");
    addModifier(game, MOD_RAIL_VALUE, -1, -1, 120, 15);
    break;

case 13:
    printf("Festival Season : Hotels receive 50%% additional rent for 5 rounds.\n");
    addModifier(game, MOD_HOTEL_RENT, -1, -1, 150, 5);
    break;

case 14:
    printf("Labour Strike : Construction suspended for 2 rounds.\n");
    game[0].economy.constructionSuspendedRoundsLeft = 2;
    break;

case 15:
    printf("Insurance Discount : Premiums reduced by 20%% for 10 rounds.\n");
    addModifier(game, MOD_INSURANCE, -1, -1, 80, 10);
    break;

case 16:
{
    PropertyGroup group;
    char groupBuffer[32];

    group = (PropertyGroup)(rand() % NO_GROUP);
    groupName(group, groupBuffer);

    printf("Property Revaluation : %s properties appreciate by 15%% for 15 rounds.\n",
           groupBuffer);
    addModifier(game, MOD_GROUP_VALUE, group, -1, 115, 15);
    break;
}

case 17:
    printf("Currency Depreciation : Construction costs increase by 10%% for 15 rounds.\n");
    addModifier(game, MOD_CONSTRUCTION, -1, -1, 110, 15);
    break;

case 18:
{
    int lucky;

    /* Section 1.3: bankrupt players take no further part in the
       game, so a grant must go to a solvent player. Pick a
       random starting point and scan forward to the first
       solvent player - at least one must exist, because a live
       player drew this card. */
    lucky = rand() % MAX_PLAYERS;

    while(game[0].players[lucky].bankrupt)
        lucky = (lucky + 1) % MAX_PLAYERS;

    printf("Government Grant : %s receives LKR 5,000.\n",

```

```

        game[0].players[lucky].name);

    receiveMoney(game, lucky, 5000);
    break;
}

case 19:
    printf("National Disaster : a random developed property is damaged.\n");
    triggerDisaster(game);
    break;

default:
    break;
}

game[0].economy.currentCardIndex = (game[0].economy.currentCardIndex + 1) % EVENT_CARD_COUNT;
}

/* Section 2.5: one of 8 events, every 15 rounds, affecting every player. */
void triggerEconomicEvent(GameState game[])
{
    int choice;

    choice = rand() % ECONOMIC_EVENT_COUNT;

    printf("\n=== Economic Event ===\n");

    switch(choice)
    {
        case 0:
            printf("Tourism Boom\n");
            printf("Hotels receive double rent for 15 rounds.\n");
            printf("Southern Province properties increase in value by 15%%.\n");

            addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 15);
            addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 115, 15);
            break;

        case 1:
            printf("Fuel Crisis\n");
            printf("Railway rent doubles for 15 rounds.\n");
            printf("Property development costs increase 20%% for 15 rounds.\n");

            addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 15);
            addModifier(game, MOD_CONSTRUCTION, -1, -1, 120, 15);
            break;

        case 2:
            printf("Heavy Monsoon\n");
            printf("Insurance premiums increase for 15 rounds.\n");
            printf("Coastal properties lose 10%% value.\n");

            addModifier(game, MOD_INSURANCE, -1, -1, 115, 15);
            addModifier(game, MOD_FLOOD_RISK, -1, -1, 100, 15);
            addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 90, 15);
            break;

        case 3:
            printf("Economic Recession\n");
            printf("Property values decrease 15%. Rent decreases 10%%.\n");
            printf("Loan interest increases by 15%%.\n");

            addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);

```

```

        addModifier(game, MOD_RENT_GLOBAL, -1, -1, 90, 15);
        addModifier(game, MOD_RECESSION, -1, -1, 100, 15);

        game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, 15);
        clampLoanInterest(game);
        break;

    case 4:
        printf("Stock Market Boom\n");
        printf("Property values increase 10%. Loan interest decreases 10%.\n");

        addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);

        game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, -10);
        clampLoanInterest(game);
        break;

    case 5:
        printf("Government Housing Programme\n");
        printf("House construction costs reduce 25%.\n");

        addModifier(game, MOD_CONSTRUCTION, -1, -1, 75, 15);
        break;

    case 6:
        printf("Foreign Investment\n");
        printf("Commercial properties increase 20%.\n");

        addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 120, 15);
        break;

    case 7:
        printf("Political Unrest\n");
        printf("Hotel rent drops by 50% for 15 rounds.\n");

        addModifier(game, MOD_HOTEL_RENT, -1, -1, 50, 15);
        addModifier(game, MOD_RIOT_RISK, -1, -1, 100, 15);
        addModifier(game, MOD_BI_CLAIMS, -1, -1, 100, 15);
        break;

    default:
        break;
}
}

/* Section 2.7: one of 8 regulations, every 20 rounds. */
void triggerGovernmentRegulation(GameState game[])
{
    int choice;

    choice = rand() % REGULATION_COUNT;

    printf("\n=== Government Regulation ===\n");

    switch(choice)
    {
        case 0:
            game[0].economy.incomeTaxRate = applyRate(game[0].economy.incomeTaxRate, 50);

            if(game[0].economy.incomeTaxRate > TAX_RATE_MAX)
                game[0].economy.incomeTaxRate = TAX_RATE_MAX;

            printf("Increase Property Tax\n");

```

```

    printf("Income Tax rate increased by 50%%. Now %d%%.\n", game[0].economy.incomeTaxRate);
    break;

case 1:
    adjustLoanInterest(game, -LOAN_INTEREST_STEP);

    printf("Reduce Loan Interest\n");
    printf("Interest decreased by %d%%. Now %d%%.\n",
        LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
    break;

case 2:
    printf("Housing Subsidy\n");
    printf("House construction costs reduced by 30%% for 20 rounds.\n");

    addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 20);
    break;

case 3:
{
    int i;
    int tax;

    printf("Luxury Property Tax\n");
    printf("Hotels pay a maintenance tax of 25%% of their value.\n");

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.hotel &&
            game[0].board[i].property.owner != -1)
        {
            tax = (currentMarketValue(game, i) * HOTEL_LUXURY_TAX_PERCENT) / 100;

            printf("%s pays LKR %d luxury tax on %s.\n",
                game[0].players[game[0].board[i].property.owner].name,
                tax, game[0].board[i].name);

            payMoney(game, game[0].board[i].property.owner, tax);
        }
    }
    break;
}

case 4:
    printf("Railway Modernization\n");
    printf("Railway rents increase 25%% for 20 rounds.\n");

    addModifier(game, MOD_RAIL_RENT, -1, -1, 125, 20);
    break;

case 5:
    printf("Electricity Tariff Revision\n");
    printf("Utility rents increase 20%% for 20 rounds.\n");

    addModifier(game, MOD_UTIL_RENT, -1, -1, 120, 20);
    break;

case 6:
    printf("Insurance Regulation\n");
    printf("Insurance premiums decrease 15%% for 20 rounds. Coverage unchanged.\n");

    addModifier(game, MOD_INSURANCE, -1, -1, 85, 20);
    break;

```

```

        case 7:
            printf("Anti-Speculation Act\n");
            printf("Players may now own at most three undeveloped properties.\n");

            game[0].economy.antiSpeculationActive = 1;
            break;

        default:
            break;
    }
}

/* Decrements the non-modifier event timers - the modifier list itself
   is ticked down separately by decrementModifiers(). */
void decrementEventTimers(GameState game[])
{
    if(game[0].economy.constructionSuspendedRoundsLeft > 0)
        game[0].economy.constructionSuspendedRoundsLeft--;

    if(game[0].economy.closedPropertyRoundsLeft > 0)
    {
        game[0].economy.closedPropertyRoundsLeft--;
        if(game[0].economy.closedPropertyRoundsLeft == 0)
            game[0].economy.closedPropertyIndex = -1;
    }
}

```